Parking Structure Management System (PSMS)

Software Architecture Design

SAD Version 3.0

Team T01
12 February 2026

Beckett Dunlavy (manager)
Aditya Chauhan
Christian Maestas
Oscar McCoy
Isaac Tapia

CS 460 Software Engineering

Table of Contents

1 Introduction

Parking structures provide concentrated parking capacity for high-demand areas such as campuses and urban centers. For such facilities to operate effectively, the system responsible for admission control and availability reporting must maintain a consistent and accurate representation of occupancy across multiple floors. In practice, availability is affected not only by vehicles occupying parking spots, but also by vehicles that have entered the structure and are traveling to locate a spot. If these intermediate conditions are not represented in the system state, the structure may admit vehicles beyond practical capacity or display incorrect availability, leading to congestion and a degraded driver experience. The quality of the software design is central to the facility's operations, as it directly impacts the accuracy of admission control and the clarity of availability reporting for drivers. Furthermore, the system is designed to prioritize safety by managing strict state transitions during emergencies or power failures.

The purpose of this Software Architecture Design (SAD) is to describe the abstractions and component interactions required to meet all stakeholder goals and regulatory requirements. This document is organized as follows. Section 2 describes the high-level design approach and includes the architecture design diagram. Section 3 provides detailed component specifications and describes their interactions. Section 4 presents a sample use case illustrating the system in operation. Section 5 outlines the design constraints, including legal and operational boundaries. Section 6 contains a glossary of technical terms used throughout this document.

2 Design Overview

This section presents the high-level architectural design of the Parking Structure Management System (PSMS). The PSMS is designed to manage a multi-floor parking garage, providing real-time tracking of vehicle entry and exit, per-spot occupancy monitoring, and automated capacity management. The architecture is driven by the requirements documented in the SRS and the design constraints identified in the RDD.

2.1 Design Approach

The PSMS employs an event-driven, centralized controller architecture. In this design, a central System Controller acts as the coordinator for all subsystem interactions, responding to real-time sensor events and orchestrating appropriate system responses. This centralized approach was chosen for two key reasons: first, the parking structure requires a single authoritative source of truth for vehicle counts and capacity status, if components operated independently, the garage could over-admit vehicles, creating safety hazards; second, the system must enforce strict state transitions (Startup, Normal Operation, At Capacity, and Emergency/Fail-Safe) that affect multiple subsystems simultaneously, which is most reliably managed through a central coordinator.

The architecture separates concerns into distinct functional components: sensor input abstraction, gate control, occupancy tracking, display management, and emergency handling. Each component communicates through well-defined interfaces with the System Controller, enabling independent testing and maintenance. The event-driven nature ensures the system reacts promptly to real-world changes, such

as a vehicle arriving at the entry gate or a power failure occurring, without relying on fixed-interval checks.

2.2 Architecture Diagram

The following architecture diagram (**Figure 1**) illustrates the software architecture of the PSMS. Software components are shown as labeled boxes, with arrows indicating the direction and nature of data flow between them. External interfaces appear at the top and bottom edges of the diagram.

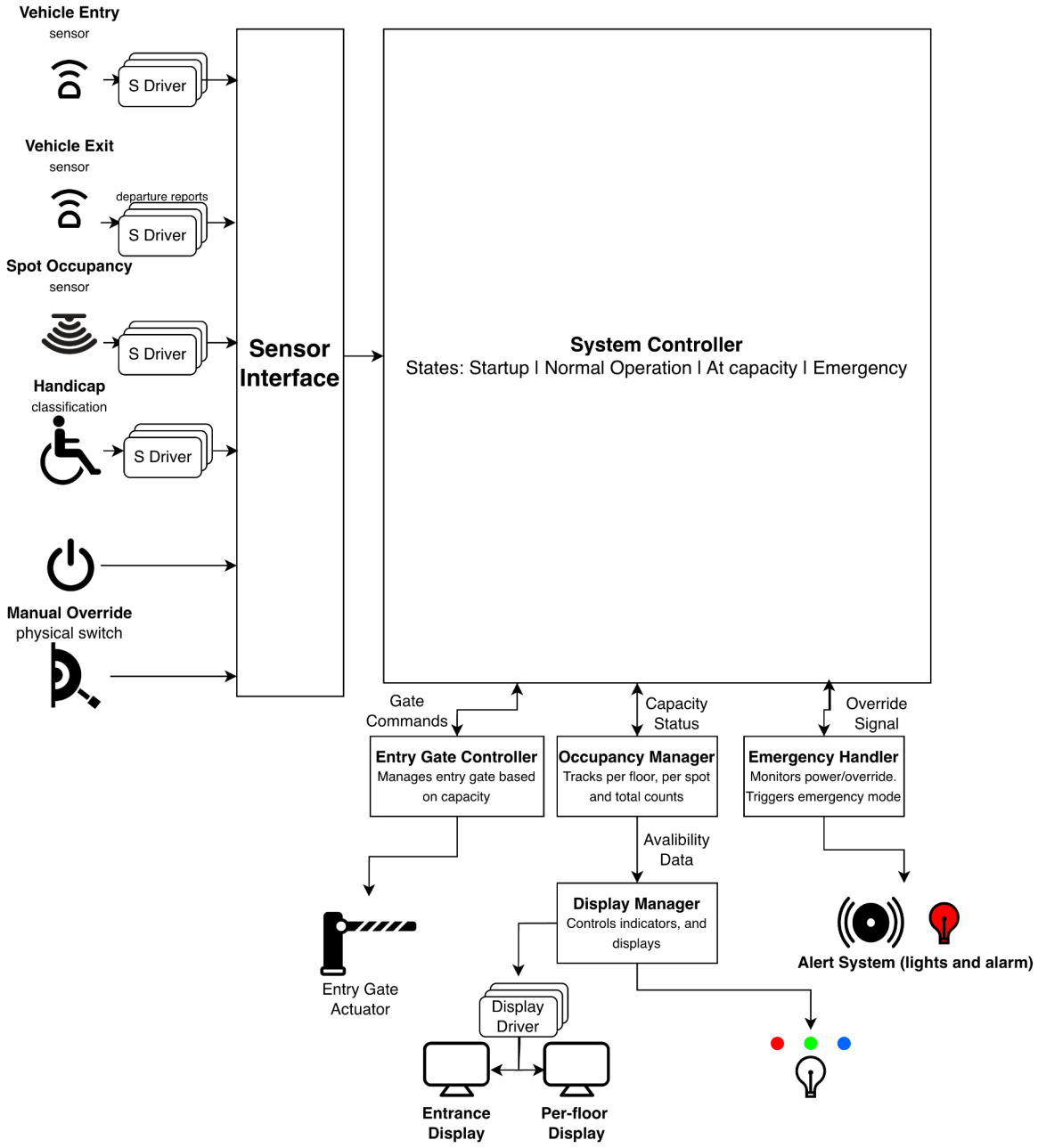

Figure 1: PSMS Architecture Diagram

2.3 Design Explanation

The architecture consists of software components and external interfaces. The following explains the role of each component and how they interact to fulfill the system requirements.

Sensor Interface

The Sensor Interface serves as the abstraction layer between all physical input devices and the rest of the software system. It receives processed signals from device drivers and binary signals: Sensor Drivers, Power Status Monitor, and Manual Override Switch. The Sensor Interface normalizes these signals into standardized event messages and routes them to the System Controller. This abstraction ensures that changes to the physical sensor hardware do not require modifications to the core system logic.

S Driver (Sensor Driver)

The S Driver is responsible for the low-level communication between the physical sensing hardware and the Sensor Interface. It takes raw electrical signals from external input sources, applies signal conditioning and noise filtering to ensure integrity, and prevents false triggers. The component then converts these validated raw signals into processed digital data and sends them to the Sensor Interface. Most importantly, the Sensor Driver abstracts away hardware-specific details like voltage thresholds and timing requirements from the core system logic.

Display Driver

The Display Driver manages the physical output of processed data onto the structure's visual devices. It translates presentation commands from the Display Manager into the specific hardware protocols required for the digital screens. For visual continuity, the display driver maintains an internal buffer reflecting the current display state to mask brief processing latency and errors. Should an input signal be disrupted or a data integrity error occur, the driver purges the buffer and initiates a restart until the signal resumes.

System Controller

The System Controller is the central coordinator of the PSMS. It implements a state machine with four primary states: System Startup, Normal Operation, At Capacity, and Emergency. Upon receiving sensor events from the Sensor Interface, the System Controller evaluates the current state, determines the appropriate response, and dispatches commands to the relevant subsystems. The System Controller is also responsible for state transitions—when the Occupancy Manager reports that the structure has reached full capacity, the System Controller transitions to the At Capacity state and instructs the Entry Gate Controller to keep the gate closed and the Display Manager to show a “FULL” message.

Entry Gate Controller

The Entry Gate Controller receives open and close commands from the System Controller and translates them into actuator signals for the physical entry gate. It enforces the access control policy: during Normal Operation, the gate opens when a vehicle is detected, and capacity is available; during the At Capacity state, the gate remains closed; and during Emergency/Fail-Safe mode, the gate locks shut to prevent new vehicles from entering the structure. The Entry Gate Controller also handles timing logic to ensure the gate remains open long enough for a vehicle to pass through before closing.

Exit Gate Sensor

The Exit Gate Controller reports each vehicle departure back to the System Controller, which in turn updates the Occupancy Manager to decrement the vehicle count.

Occupancy Manager

The Occupancy Manager is responsible for maintaining the count of all vehicles within the parking structure. It operates at three levels of granularity: per-spot, per-floor, and total structure capacity. The Occupancy Manager also tracks vehicles in the "in-transit" state—those that have entered through the gate but have not yet parked in a spot. It distinguishes between regular and handicap-designated spaces based on the Handicap Classification input. The Occupancy Manager feeds data to both the System Controller and the Display Manager. When the total available count reaches zero, it notifies the System Controller to trigger the At Capacity state transition.

Display Manager

The Display Manager controls all visual output devices in the parking structure. It receives availability data from the Occupancy Manager and drives three categories of displays: overhead Spot Indicator Lights, Per-Floor Digital Displays, and the Entrance Availability Display. The Display Manager updates these outputs in real time as occupancy changes occur, providing immediate visual feedback to drivers navigating the structure.

Emergency Handler

The Emergency Handler monitors two critical inputs: the Power Status Monitor and the Manual Override Switch. When a power failure is detected or a manual override is activated, the Emergency Handler immediately signals the System Controller to transition to the Emergency/Fail-Safe state. In this state, the entry gate locks are closed, and all non-essential display operations are suspended. The Emergency Handler operates with the highest priority in the system to ensure that safety-critical responses are never delayed by normal processing. Once the emergency condition is resolved, the Emergency Handler signals the System Controller to begin a return to Normal Operation through the Startup state.

3 Component Specifications

The following section provides a detailed breakdown of the software components noted in the PSMS architecture. Each specification defines the internal state, or Variables, and the operational capabilities, or Methods, required for the component to fulfill its role within the system. These specifications serve as the blueprint for the implementation phase. These specifications ensure that data flow from the initial sensor detection to the real-time display updates is handled through standardized interfaces.

Sensor Interface

Variables:

- RawSignal currentSignal; The most recent signal received from physical hardware.
- Event normalizedEvent; The standardized message format for the system controller.

Methods:

- public Event normalizeSignal(RawSignal) { convert hardware voltage/data to System Event }
- public void routeToController(Event) { send event to System Controller }

System Controller

Variables:

- SystemState currentState; Enum for STARTUP, NORMAL, AT_CAPACITY, or EMERGENCY.

Methods:

- public void transitionState(SystemState) { logic to switch states and updated system flags }
- public void processEvent(Event) { evaluate the event based on currentState, trigger subsystem commands }

Entry Gate Controller

Variables:

- boolean isGateOpen; Status of the physical entry gate.

Methods:

- public void openGate() { send open signal to actuator }
- public void closeGate() { send close signal to actuator }
- public void handleAccesses(SystemState) { determine if the gate should open based on current occupancy state }

Exit Gate Sensor

Variables:

- boolean vehicleDetected; Status of the exit sensor.

Methods:

- public void reportDeparture() { trigger the System Controller to decrement the occupancy count }

Occupancy Manager

Variables:

- int totalCapacity; Maximum number of vehicles allowed.
- int currentCount; Current number of vehicles in the structure.
- int handicapCount; Current number of occupied handicap spots.
- int inTransitCount; Current number of vehicles in transit, vehicles past the gate but not parked.

Methods:

- public void updateCount(boolean isEntering, boolean isHandicap) { increment or decrement counts }
- public boolean isFull() { return true if currentCount >= totalCapacity }
- public int getFloorAvailability(int floor) { return remaining spots for each specific floor }

Display Manager

Variables:

- String entranceMessage; Current text for main entry sign (“100 Spots Available”, “FULL”).

Methods:

- public void updateStallLights(StallID, Color) { change LED color for specific parking spots }
- public void updateFloorDisplay(int floor, int count) { update digital readout on each floor }
- public void setEntranceSign(String message) { update the main sign at the entry gate }

Emergency Handler

Variables:

- boolean powerFailure; Top-priority flag from the Power Status Monitor.
- boolean manualOverride; Status of the physical override switch.

Methods:

- public void monitorInputs() { poll the power and manual switch status }
- public void triggerEmergency() { immediately signal the System Controller to enter Fail-Safe mode }
- public void resolveEmergency() { signal the System Controller to initiate the Startup sequence }

4 Sample Use Cases

The seven included use cases, such as System Startup, Vehicle Entry, Vehicle Exit, and Emergency State Activation, were selected to illustrate all primary operational states and critical transitions of the Parking Structure Management System (PSMS). This set covers the full system life cycle, from normal operation to error handling and safety-critical functions. Collectively, these scenarios are illustrative of the majority of real-world situations, ensuring that all key system components interact with the central System Controller to fulfill requirements.

4.1 System Startup

This use case describes the automated initialization of the system components and the resumption of monitoring and admission control.

Primary Actor: System

Goal: To reactivate all hardware interfaces and transition to an active operational state.

Preconditions: The system has been powered on or has completed the Emergency State Resolution sequence.

Trigger: The System Controller enters the Startup state.

Scenario:

1. The System initializes internal software modules and synchronizes current occupancy counts with the physical sensor network. (**processEvent(), updateCount(), isFull()**)
2. The per-floor digital displays and overhead spot indicator lights activate to reflect the current occupancy state. (**updateFloorDisplay(), updateStallLights(), setEntranceSign()**)
3. The System performs a hardware check to ensure the entry gate is in the closed and locked position. (**closeGate()**)
4. The System transitions to the Normal Operation state to begin standard admission control. (**transitionState()**)

4.2 Vehicle Entry Under Normal Operation

This use case describes the standard process of a vehicle arriving at the parking structure and being admitted based on current capacity.

Primary Actor: Driver

Goal: To enter the parking structure and locate an available parking space.

Preconditions:

- The PSMS is in the Normal Operation state
- The structure-level occupancy has not reached Total Capacity.

Trigger: A vehicle is detected by the Vehicle Entry Sensor**.**

Scenario:

1. The driver approaches the entrance and is detected by the entrance sensor. (**normalizeSignal(), routeToController(), processEvent(), isFull(), handleAccesses(), openGate()**)
2. The entry gate opens, and the vehicle enters the parking structure. (**updateCount(), closeGate(), setEntranceSign(), updateFloorDisplay()**)
3. The driver identifies a vacant parking stall via a green indicator light and parks the vehicle. (**updateCount(), updateStallLights(), updateFloorDisplay()**)

4.3 Vehicle Entry Under Full Capacity

This use case describes the system behavior when a vehicle attempts to enter the structure while it is at maximum capacity, and the subsequent admission once a space is vacated.

Primary Actor: Driver

Goal: To enter the parking structure after a space becomes available.

Preconditions:

- The PSMS is in the Full state.
- The structure-level occupancy has reached Total Capacity
- The entrance sign displays “FULL”

Trigger: A vehicle is detected by the Vehicle Entry Sensor.

Scenario:

1. The driver approaches the entrance and is detected by the entrance sensor. (**normalizeSignal(), routeToController(), processEvent(), isFull(), handleAccesses()**)
2. The driver waits for a vehicle to exit.
3. A vehicle exits the parking structure (**normalizeSignal(), routeToController(), processEvent(), reportDeparture(), updateCount(), setEntranceSign()**)
4. The entry gate opens, and the waiting vehicle enters the parking structure. (**openGate(), updateCount(), closeGate(), setEntranceSign(), updateFloorDisplay()**)
5. The driver identifies a vacant parking stall via a green indicator light and parks the vehicle. (**updateCount(), updateStallLights(), updateFloorDisplay()**)

4.4 Vehicle Exit

This use case describes the process of a vehicle vacating a parking stall and successfully exiting the parking structure, resulting in an update to the system's occupancy state.

Primary Actor: Driver

Goal: To vacate a parking spot and exit the facility.

Preconditions:

- The PSMS is in the Normal Operation or Full state.
- The vehicle is currently occupying a parking stall within the structure.

Trigger: A vehicle vacates a parking stall.

Scenario

1. The driver vacates the parking stall and begins driving toward the exit. (**updateCount(), updateStallLights(), updateFloorDisplay()**)
2. The driver passes the exit sensor as they leave the parking structure. (**normalizeSignal(), routeToController(), processEvent(), reportDeparture(), updateCount(), setEntranceSign()**)

4.5 Vehicle Relocation

This use case describes the process of a vehicle vacating one parking stall and successfully parking in a different stall within the structure, resulting in a sequence of updates to internal floor and spot-level counts without an overall structure-level change.

Primary Actor: Driver

Goal: To move the vehicle from one parking stall to another within the parking structure without exiting.

Preconditions:

- The PSMS is in the Normal Operation state.
- The vehicle is currently parked in a stall (Stall A).
- At least one other parking stall is currently available.

Trigger: The driver vacates Stall A.

Scenario

1. The driver vacates Stall A and drives to find another parking spot. (**updateCount(), updateStallLights(), updateFloorDisplay()**).
2. The driver identifies a vacant parking stall (Stall B) via a green indicator light and parks the vehicle. (**updateCount(), updateStallLights(), updateFloorDisplay()**)

4.6 Emergency State Activation

This use case describes the high-priority transition of the system into the Emergency/Fail-Safe state when a power failure is detected or a manual override is triggered.

Primary Actor: System Administrator / Power Status Monitor

Goal: To immediately secure the facility and suspend standard operations.

Preconditions: The PSMS is in any operational state (Normal Operation, Full, or Startup).

Trigger:

- The System Administrator activates the Manual Override Switch.
- The Power Status Monitor detects a loss of electrical power.

Scenario

1. The System Administrator activates the manual override switch, or the Power Status Monitor detects a loss of power. (**monitorInputs()**).
2. The administrator observes the entry gate move to a locked position, and all stall indicator lights and availability displays deactivate. (**triggerEmergency()**)

4.7 Emergency State Resolution

This use case describes the procedure for exiting the Emergency/Fail-Safe state and preparing the system for a return to standard functionality.
Primary Actor: System Administrator
Goal: To terminate the emergency mode and initiate the transition back to normal operations.
Preconditions:

- The PSMS is in the Emergency/Fail-Safe state.
- The external condition (Power Failure or Manual Override) has been physically resolved.

Trigger: The System Administrator deactivates the Manual Override Switch.
Scenario

1. The System Administrator deactivates the physical manual override switch. (**monitorInputs()**)
2. The administrator observes the system initiating the startup sequence to resume normal tracking and operations (**resolveEmergency()**)

5 Design Constraints

The following constraints define the technical boundaries and performance requirements for the PSMS software architecture. These design constraints specifically limit software development and operational logic.

Real-Time Performance and Latency
The system must process sensor events and update physical actuators within strict time limits to ensure safety and a smooth user experience.

- **Gate Response Time**: The Entry Gate Controller must issue an "Open" or "Close" command within a quick enough time of the System Controller receiving a normalized vehicle detection event.
- **Display Synchronization**: Availability data must be updated across all categories of displays (Entrance, Per-Floor, and Spot Indicators) within a small time frame of a state change in the Occupancy Manager.
- **Event Processing**: The Sensor Interface must normalize raw signals into standardized event messages without queuing delays that are too long.

Power Failure and Data Integrity
The software must maintain state consistency during and after power interruptions.

- **Non-Volatile State Persistence**: The Occupancy Manager must periodically persist the currentCount and inTransitCount to non-volatile memory to prevent data loss during a power failure.
- **Emergency Transition Priority**: Upon receiving a signal from the Power Status Monitor, the Emergency Handler must trigger the EMERGENCY state transition quickly, overriding all other system processes.
- **Post-Power Standby**: After power restoration, the system is constrained by a mandatory 10-minute safety standby period before transitioning from STARTUP to NORMAL OPERATION.

Reliability and Error Detection
The software architecture must account for the inherent limitations and potential failures of the physical sensor network.

- **Sensor Fault Tolerance**: The System Controller must implement logic to identify "stuck" sensors (e.g., a spot marked occupied for 72+ hours) and flag them for manual inspection without crashing the overall occupancy logic.
- **In-Transit Logic Bounds**: Because the "In-Transit" count is a software-calculated value, the Occupancy Manager must include a reconciliation function to prevent negative counts caused by missed sensor triggers.

Operational State Constraints

- **Centralized Authority**: All subsystem interactions must be orchestrated by the System Controller; components are strictly forbidden from autonomous state changes to ensure a single authoritative source of truth for capacity.
- **Fail-Safe Defaults**: In the absence of a valid signal from the Sensor Interface, the Entry Gate Controller must default to a "Closed/Locked" state.

6 Definition of Terms

Admission Control
The logic that determines whether a vehicle is permitted to enter the parking structure based on current occupancy and system state.

At Capacity
A system state in which the total number of vehicles inside the structure equals the maximum defined capacity, preventing additional vehicle entry.

Binary Spot Sensor
A parking stall sensor that reports only two possible states: *occupied* or *vacant*.

Emergency / Fail-Safe State
A high-priority operational state triggered by power failure or manual override, during which entry is locked, and non-essential system functions are suspended to ensure safety.

Entrance Availability Display
The main display at the parking structure entrance that shows real-time availability information (e.g., “FULL” or “25 Spaces Available”).

Event (System Event)
A standardized message generated by the Sensor Interface representing a detected condition (e.g., vehicle entry, vehicle exit, stall occupancy change).

In-Transit Vehicle
A vehicle that has entered through the gate but has not yet occupied a parking stall, or a vehicle that has vacated a stall but has not yet exited the structure.

Manual Override Switch
A physical administrative control used to manually force the system into Emergency/Fail-Safe mode.

Normal Operation State
The standard operating condition of the PSMS is one in which vehicles may enter and exit based on availability and occupancy logic.

Occupancy Manager
The subsystem is responsible for tracking vehicle counts at the per-stall, per-floor, and total-structure levels.

Overhead Spot Indicator Light
An LED-based visual indicator above a parking stall that displays stall availability (commonly green for vacant, red for occupied).

Power Status Monitor
A hardware interface that detects electrical power failures and reports status changes to the Emergency Handler.

Startup State
A temporary system state during initialization or recovery from an emergency, in which subsystems are reactivated and synchronized before transitioning to Normal Operation.